University of Tehran
College of Interdisciplinary Science and Technologies

Soft Computing

4th Assignment

Name: Mahan Madani
Student ID: 830504035

Contents

List of Figures

List of Tables

Question 1. Research

1.1 t-SNE Algorithm

t-distributed Stochastic Neighbor Embedding (t-SNE) is an unsupervised nonlinear dimensionality reduction algorithm primarily used for **data visualization and exploration** in low-dimensional spaces, usually 2D or 3D.

The main idea of t-SNE is to preserve local neighborhood structure. Given high dimensional data points, t-SNE converts pairwise distances into conditional probabilities that represent similarities. Using these probabilities, t-SNE tries to keep similar data points close together, and push data points that are different further apart.

The use of the t-distribution helps alleviate the **crowding problem**, where points that are moderately distant in high dimensions would otherwise be forced too close together in low dimensions.

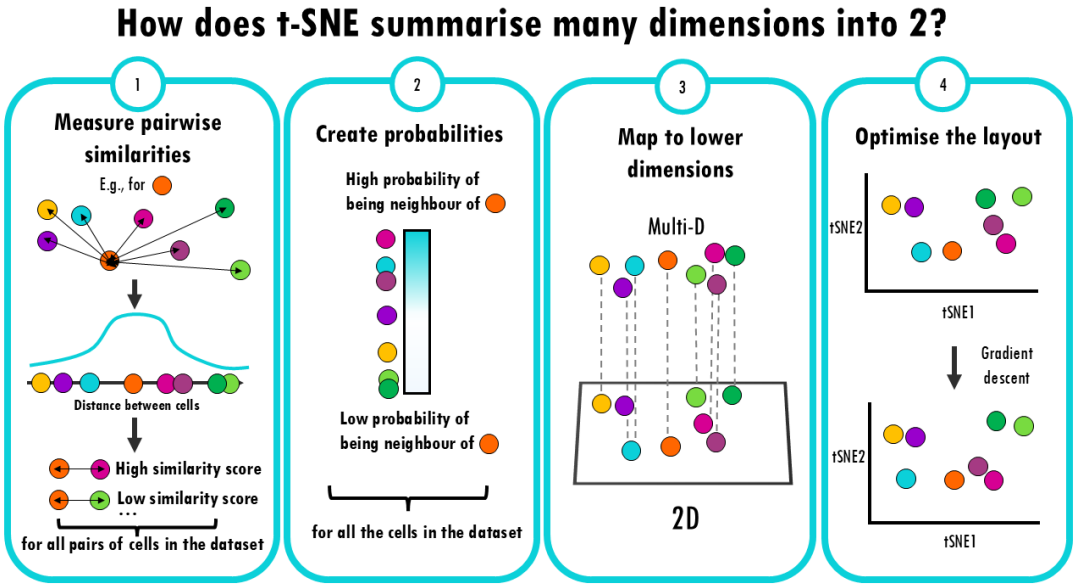


Figure 1: A high-level explanation of the t-SNE algorithm.

Figure 1 showcases the general idea behind t-SNE. In the 4th step, t-SNE optimizes the layout of the data in the lower-dimensional space using gradient descent. The goal is to make the distribution of similarities in the lower space match the original distribution as closely as possible.

In order to achieve this, t-SNE minimizes the **Kullback–Leibler (KL) divergence** between the similarity distributions in the high-dimensional and low-dimensional spaces:

$$\mathrm{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}},$$

where p_{ij} and q_{ij} denote pairwise similarities in the original and embedded spaces, respectively.

t-SNE Applications

- Visualization of high-dimensional data such as word embeddings (e.g., GloVe).
- Exploring latent spaces of deep learning models (e.g., autoencoders, VAEs).
- Visual analysis of classification datasets to observe class clustering.

1.2 PCA Algorithm

Principal Component Analysis (PCA) is a linear dimensionality reduction technique that transforms data into a new vector space such that the directions (principal components) maximize variance.

Given a centered dataset $X \in \mathbb{R}^{n \times d}$, PCA finds orthogonal directions $\{w_1, w_2, \dots\}$ by solving an eigenvalue problem on the covariance matrix:

$$\Sigma = \frac{1}{n} X^\top X.$$

The principal components are the eigenvectors of Σ corresponding to the largest eigenvalues. These eigenvectors form the basis of the new vector space.

The first principal component captures the maximum variance in the data, the second captures the maximum remaining variance orthogonal to the first, and so on. **Figure 2** displays an example of finding the principal components of a simple dataset.

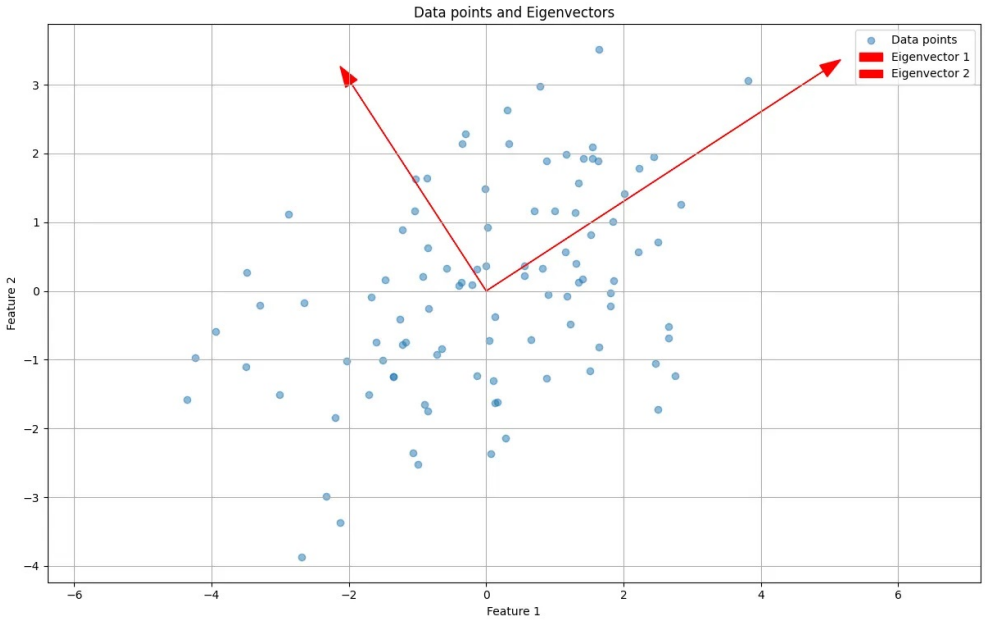


Figure 2: Plotting the eigenvectors of a sample dataset with 2 features.

PCA Applications

- Data compression and dimensionality reduction.
- Noise reduction and feature decorrelation.
- Visualization of high-dimensional datasets in 2D or 3D.

1.3 Comparison of t-SNE and PCA

Factor	t-SNE	PCA
Linearity	Nonlinear dimensionality reduction method	Linear dimensionality reduction method
Approach	Preserves local neighborhood relationships	Preserves global variance and overall data geometry
Mathematical basis	Probability distributions and minimization of KLD	Eigenvalue decomposition of the covariance matrix
Interpretation	embedded axes have no direct meaning	components are linear combinations of the original features
Performance	Computationally expensive and slower on large datasets	Computationally efficient and scalable to large datasets
Stability	Sensitive to hyperparameters and random initialization	Deterministic with a unique solution for a given dataset
Use cases	Data visualization and exploratory analysis of high-dimensional data	Preprocessing, compression, noise reduction, and feature extraction
Downstream tasks	Generally not suitable due to loss of global structure	Well-suited as input to machine learning models

Table 1: Comparison of t-SNE and PCA

Summary: PCA is preferred when interpretability, efficiency, and global structure matter, while t-SNE is preferred when visualizing high-dimensional data with meaningful local clusters.

1.4 Hate Detection Modeling

We are given a dataset of tweets, each consisting of an image and an associated text. The goal is to design a network that models can detect whether the tweet contains hate-related content, such as racism or sexism. This is a multimodal problem, as we are working with both text and image data (two modalities).

1.4.a Challenges in Modeling This Problem

Data Labeling

The dataset must be labeled, as this is a supervised classification task. Because Labeling hate content is inherently subjective and depends on cultural norms, this can introduce noise into the dataset. Additionally, Hate speech can be implicit, using irony, sarcasm, metaphors, or cultural references. This makes it difficult for all data points to be labeled correctly.

Data Multimodality Challenges

Finding an appropriate architecture for handling Multimodal data can be a challenge, as there are multiple approaches available. The text and image components of the

tweet can be embedded separately, or a multimodal model like **CLIP** can be used to find embeddings for both modalities in the same space. Either approach may produce decent results, so ideally, both should be tested.

Semantic Gap Between Image and Text

Hate speech may not be present in the image or the text alone, but rather emerge from their combination. For example, a neutral image combined with hateful text. Our model should be able to learn the correct correlations between images and texts with hate-speech, otherwise it might label tweets incorrectly soley based on the text or image embedding.

Data Imbalance

Hate-related samples are typically much fewer than non-hate samples, leading to class imbalance and biased learning.

1.4.b Proposed Architecture

To model the relationship between images and text in tweets, we can use a **CLIP-based multimodal architecture**. CLIP (Contrastive Language–Image Pretraining) is extremely suitable for this task as it embeds images and text into a shared semantic space, allowing for cross-modal reasoning.

1. Input Processing

Each data sample consists of an image extracted from the tweet, and the corresponding tweet text. The image is resized and normalized according to CLIP preprocessing standards. The text is cleaned (handling emojis, hashtags, etc.) and tokenized using CLIP's text tokenizer.

2. CLIP Image Encoder

The image is passed through CLIP's pretrained vision encoder (Vision Transformer-based). This encoder extracts high-level visual features. The output is a fixed-length image embedding representing the semantic content of the image. The CLIP encoder can be **fine-tuned** to produce more accurate results for our dataset.

3. CLIP Text Encoder

The tweet's text is processed by CLIP's pretrained text transformer. This encoder produces an embedding that captures the semantic meaning of the text. The text output is mapped into the same embedding space as the image embedding. Similar to the vision encoder, the text encoder can also be **fine-tuned** to produce more accurate results for our dataset.

4. Multimodal Representation MLP

The image and text embeddings are combined to form a joint multimodal representation. The embeddings can be concatenated or added together, then fed to a small MLP that learns cross-modal interactions from the joint embeddings.

5. Classification Head

The fused multimodal representation is passed through the MLP and fed to the final layer of the model, outputting a binary prediction (using softmax) indicating whether the tweet contains hate speech.

The final architecture is: Data → CLIP Encoder → MLP → Classification Output

Question 2. Image Captioning with RNN

The goal of the question is to implement a recurrent neural network (RNN) to train a model that can generate natural language captions for images.

The code required by this question is implemented in `rnn_captioning.py` , and the experiments are run in the `rnn_captioning.ipynb` file.

2.1 Dataset and Data Preprocessing

The dataset used for this question is the 2014 release of the COCO Captions dataset, which consists of 80,000 training images and 40,000 validation images, each annotated with 5 captions. **Figure 3** displays a sample image alongside one caption.

The preprocessed version of the data was provided for this question. It contains 10,000 image-caption pairs for training and 500 for testing. The images have been downsampled to 112x112 for computation efficiency, and captions are tokenized and numericalized, clamped to 15 words.

A number of special tokens are added to the vocabulary:

- `<START>` , `<END>` : Tokens added to the beginning and end of each caption.
- `<UNK>` : Rare words are replaced with this token (Unknown).
- `<NULL>` : Used for padding short captions. Loss and gradient are not computed for this token.

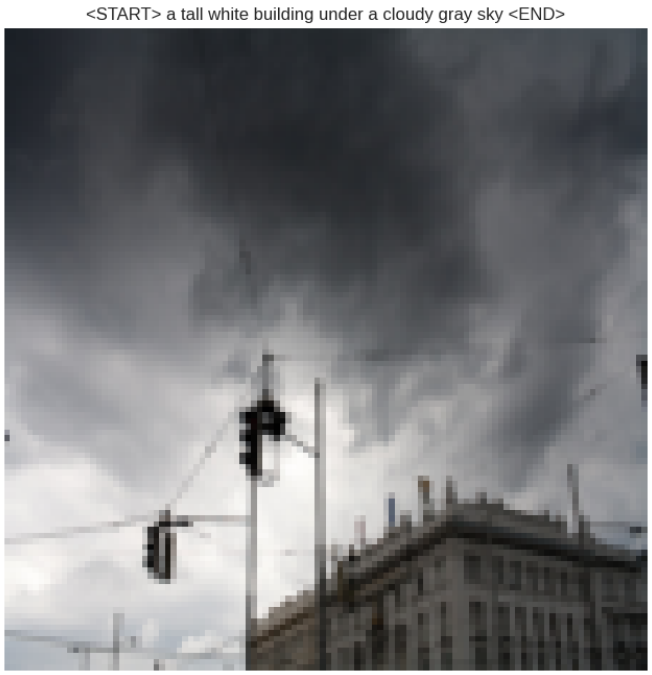


Figure 3: A sample of the training data used for image captioning.

2.2 Captioning RNN Architecture

The `CaptioningRNN` class uses the various functions and classes created to generate captions for images. It consists of:

- **Image Encoder**: The RegNet-X 400MF model is used to extract a feature vector for the images. This network is a CNN initialized with ImageNet-pretrained weights.
- **Word Embedding:** The tokenized captions are embedded using an embedding matrix, with the shape `(vocab_size, embed_size)`. The embedding matrix acts as a lookup table; when a word index is passed to it, it retrieves the corresponding rows (embedding vectors) from the matrix. This matrix is learned during training.
- **RNN**: The RNN module utilizes the `rnn_forward` and `rnn_backward` functions, which are implemented from scratch using PyTorch Tensor operations. The RNN receives the word vectors (from the word embedding layer) and the image feature vectors (from the image encoder) to generate captions for the images.

Note that **Temporal Softmax** is used as the loss function in RNNs, which is a variant of softmax applied independently at each time step of a sequence produced by an RNN. Temporal softmax computes a probability distribution over the vocabulary for every time step in the sequence. This allows the model to predict each position (each word in a sentence).

Figure 4 displays the training loss of the full captioning model, trained over 60 epochs, using the AdamW optimizer and a learning rate of 0.001.

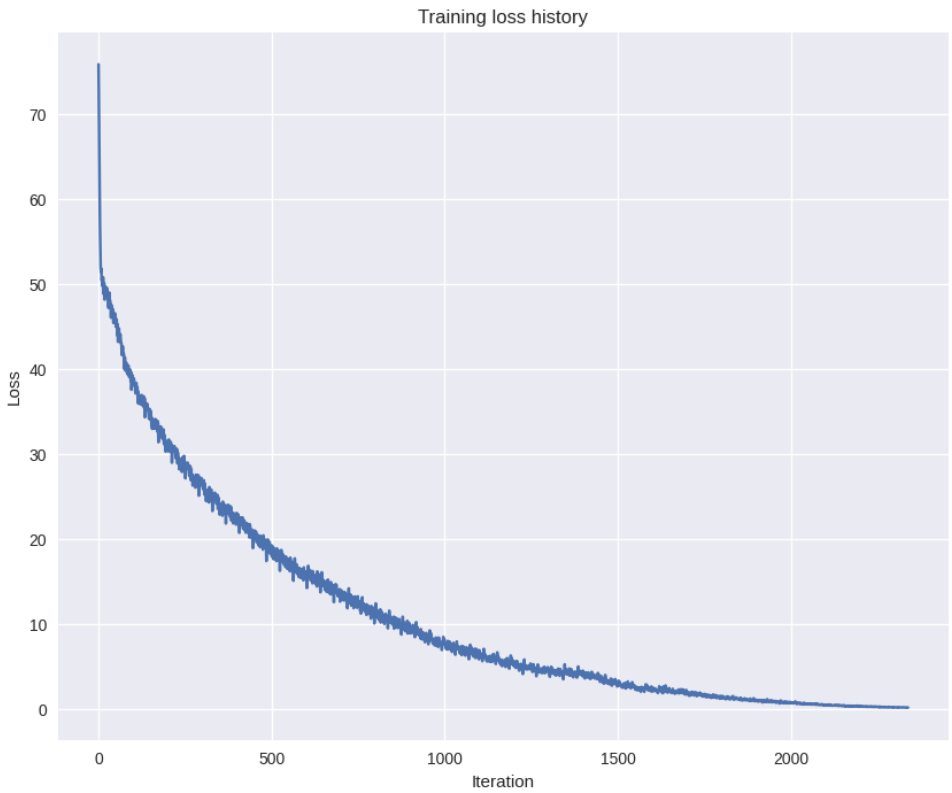


Figure 4: Training loss plot of the Captioning RNN.

2.3 Image Captioning Results

For test time sampling, the Captioning RNN model generates captions sequentially by relying on its own predictions. The initial hidden state is computed from the image features, and the process starts with the <START> token as input.

At each timestep, the current word is embedded and passed with the previous hidden state into the RNN to produce a new hidden state, which is then used to score all words in the vocabulary. **The next word is chosen from these scores** and fed back into the RNN for the following timestep. This continues until an <END> token is produced, or a maximum length is reached. This is different from the training process where ground-truth words are provided at each step.

Figure 5: Captioning for sample image from the **training** dataset.

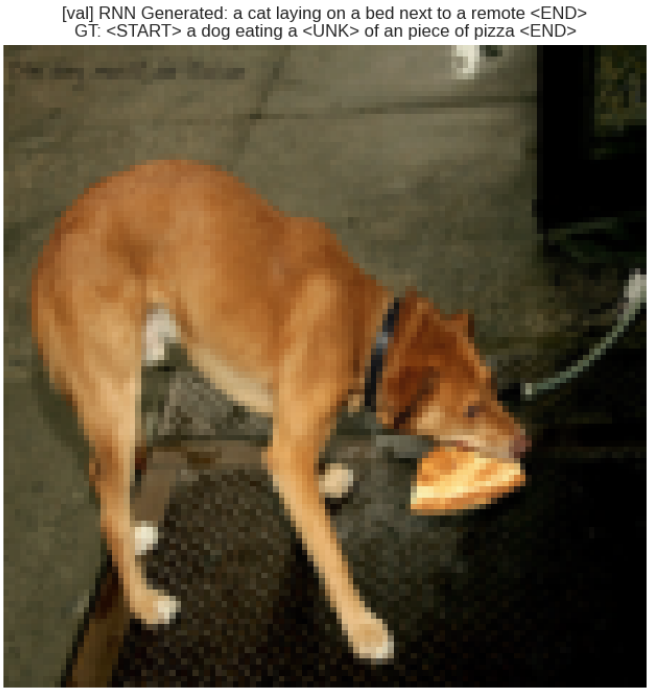


Figure 6: Captioning for sample image from the **validation** dataset.

Figure 5 displays a sample image from the **training** dataset, alongisde its ground truth caption and the caption generated by the model. We can see that the generated caption is quite accurate and matches the image.

Figure 6 displays a sample image from the **validation** dataset, alongisde its ground truth caption and the caption generated by the model. The caption doesn't match the image accurately, but it still managed to identify an animal, eventhough cat $\neq$ dog.

These results (alongside other samples found in the code file) indicate our model has managed to learn the training data effectively but **lacks the generalization** required for captioning unseen images.

Question 3. Fake News Detection with LSTM

The goal of the question is to implement a Hybrid CNN + LSTM to train a model to classify news articles as fake or real, as proposed in the **'Fake news detection: A Hybrid CNN-RNN based deep learning approach'** paper.

We also compare the results of the hybrid model to an LSTM-only model to showcase the feature extraction capabilities of CNNs, even when used on text data.

The code required by this question is implemented in `lstm.ipynb`, using the TensorFlow library, as this is the library used in the source paper.

3.1 Model Description and Comparison

A **standard RNN** processes a sequence by maintaining a hidden state that is updated as each word in the sequence is read, allowing the model to retain some memory of previous inputs. Simple RNNs are affected by **vanishing gradients**, making it hard to capture information from distant words in long sentences.

Long Short-Term Memory (LSTM) networks are a specialized type of RNN that address this issue by introducing gated memory cells. These gates (input, forget, output) control the flow of information, enabling the network to retain information over much longer sequences.

This makes LSTMs more effective at modeling the context and dependencies in text sequences, which is needed fort for natural language tasks where meaning often depends on the relationship between words far apart in the sentence.

The **Hybrid model** proposed in the paper combines a CNN feature extractor with an LSTM layer to leverage the strengths of both architectures. The CNN extracts **local patterns** in the text as features, and the LSTM processes these features to capture **temporal relationships** across the sentence.

Compared to using a plain recurrent model alone, the Hybrid model can learn local features (benefit of using the CNN features) and long-term dependencies, resulting in higher accuracy in many classification tasks.

The full architecture of the Hybrid model is displayed in **Figure 7**, implemented using TensorFlow

Model: "Hybrid"

Layer (type)	Output Shape	Param #
embedding_4 (Embedding)	(None, 300, 100)	1,017,900
conv1d_5 (Conv1D)	(None, 296, 128)	64,128
max_pooling1d_5 (MaxPooling1D)	(None, 148, 128)	0
lstm_10 (LSTM)	(None, 32)	20,608
dense_10 (Dense)	(None, 1)	33

Figure 7: Hybrid CNN + LSTM model architecture.

3.2 Word Embedding

When text data is used as input to a neural network, the raw words must be converted into a numerical form that the model can process. **Word embedding** is a representation technique that maps words into vectors in a low-dimensional space, where the semantic relationships between words are preserved.

Unlike one-hot encoding, word embeddings capture similarity (words with similar meanings have vectors that are close to each other), which is essential for effective text modeling.

Note that each word in the text must first be **tokenized** and mapped to an index in a vocabulary. These indices are then passed to an embedding layer, which converts them into vectors:

$$\text{text sequence} \rightarrow \text{sequence of tokens} \rightarrow \text{sequence of embedding vectors}$$

Glove Embedding Method

For this project, **Global Vectors for Word Representation (GloVe)** embeddings are used (**Embedding Dimension = 100**).

Pretrained GloVe vectors are utilized to initialize the weights of the **embedding layer** in the model. This allows the network to start with better representations of words rather than learning them from scratch. During training, these embeddings are fine-tuned to better adapt to the dataset.

3.3 Preprocessing Text Data

The dataset used has multiple features (including article_title, article_content, source, date, location, labels), but we will only use the text data in **article_content** as the training set and **labels** as the target feature.

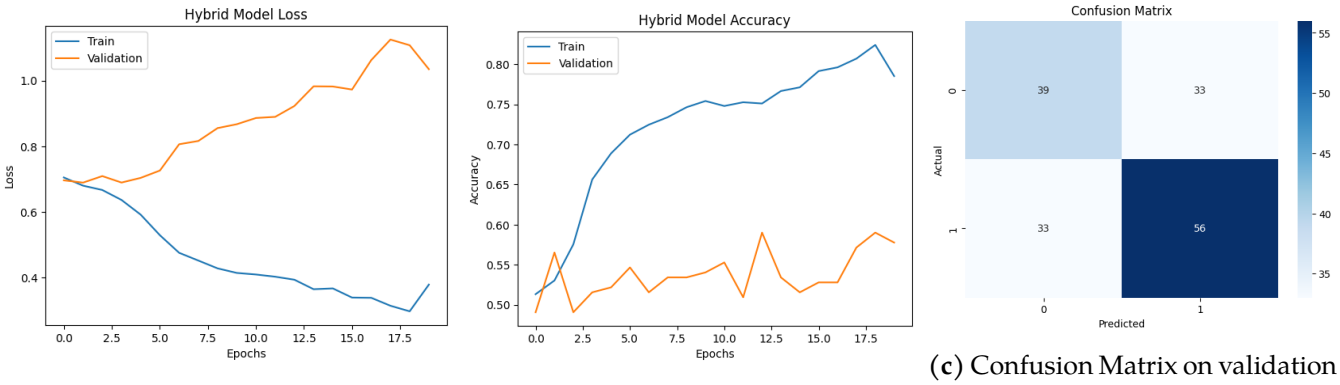
The text data is tokenized using the default TensorFlow tokenizer, which automatically creates the vocabulary for the tokens. We set **Max Sequence Length = 300**. Sequences shorter than 300 tokens are padded with zeros to reach the sequence limit, and sequences longer than the limit are truncated.

The tokenized sequences are then fed to the model, where the first layer (Embedding Layer) is initialized with the weights from the GloVe embedding matrix.

3.4 Training the Hybrid and LSTM-only Models

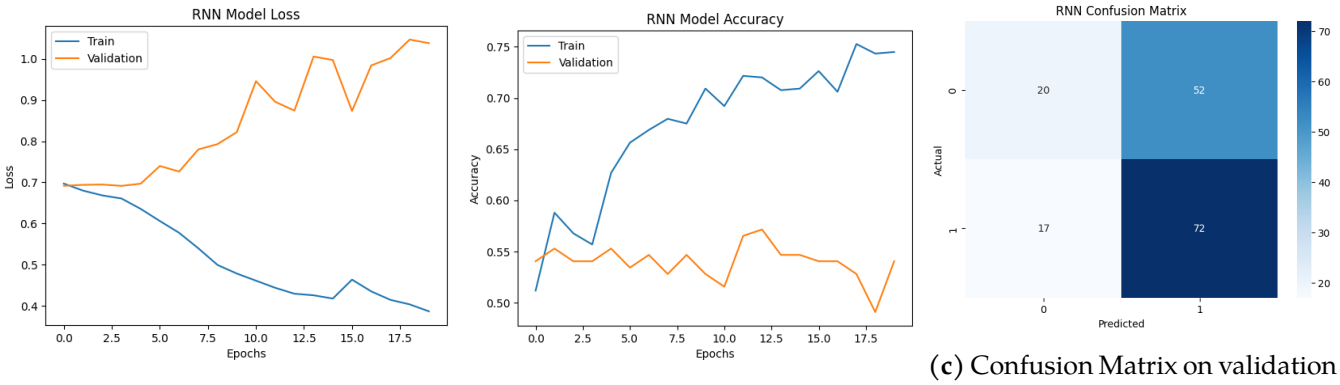
The dataset is split into training and validation sets using an 80/20 split ratio. The two created models (Hybrid CNN+LSTM and LSTM-only) are trained with the following hyperparameters:

- Epochs = 20
- Batch Size = 32
- Optimizer = Adam
- Learning Rate = 0.01



(a) Loss over epochs (b) Accuracy over epochs (c) Confusion Matrix on validation data

Figure 8: Hybrid Model Loss, Accuracy, and Confusion Matrix



(a) Loss over epochs (b) Accuracy over epochs (c) Confusion Matrix on validation data

Figure 9: LSTM-only Model Loss, Accuracy, and Confusion Matrix

Model	Accuracy	Precision - 0	Precision - 1	Recall - 0	Recall - 1	f1 - 0	f1 - 1
Hybrid	0.59	0.54	0.63	0.54	0.63	0.54	0.63
LSTM	0.57	0.54	0.58	0.28	0.81	0.37	0.68

Table 2: Comparison of Accuracy, Precision, and Recall for the Hybrid and LSTM-only Model on the validation data

3.5 Results Analysis

Based on **Figure 8**, **Figure 9**, and **Table 2**, we can see how the Hybrid model achieves better results than the LSTM-only model as it reaches better accuracy values, but more importantly, the LSTM-only model is biased towards labeling most data points as 1 (fake), while the Hybrid model has significantly higher recall for data points with the ground truth label 0.

To further improve accuracy, several strategies could be applied. Using more advanced embedding like **BERT** can help the model better understand semantic nuances. Adding **Attention mechanisms** on top of the LSTM could help the model focus on critical words or phrases that indicate misinformation. Additionally, integrating the **other features** in the dataset as part of the training data could further enhance detection.

Question 4. Image Generation with VAE

The goal of the question is to implement a VAE model to generate sample images of cats. The provided dataset is comprised of 29843 close-up images of cats.

The code required by this question is implemented in `vae.ipynb`, using the PyTorch library.

4.1 VAE Loss

The loss function of a Variational Autoencoder (VAE) consists of two main components: a **reconstruction loss** and a **regularization loss**. Together, these terms encourage the model to both accurately reconstruct the input data and learn a smooth, well-structured latent space.

The reconstruction loss measures how well the decoder reconstructs the input x from the latent variable z. Depending on the data type, this term is commonly implemented as binary cross-entropy or mean squared error.

The regularization term is the Kullback–Leibler (KL) divergence between the learned posterior distribution $q_\phi(z|x)$ and a prior distribution $p(z)$, usually a standard normal distribution $\mathcal{N}(0, I)$. This term enforces the latent space to follow a known distribution.

The total VAE loss is given by:

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q_\phi(z|x)}\big[-\log p_\theta(x|z)\big] + \text{KL}\big(q_\phi(z|x) \,\|\, p(z)\big)$$

The first term promotes accurate reconstruction, while the second term regularizes the latent representation.

4.2 Reparameterization Trick

The reparameterization trick is a technique used in VAEs to allow gradient-based optimization when sampling from a probability distribution. Directly sampling $z \sim q_\phi(z|x)$ is not differentiable with respect to the encoder parameters ϕ, which prevents backpropagation.

To solve this, the random sampling operation is rewritten as a deterministic function of the parameters and an independent random noise variable. For a Gaussian posterior, this is done as:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Here, the randomness is isolated in ϵ, which does not depend on the model parameters. This transformation makes the sampling process differentiable, enabling end-to-end training of the VAE using standard backpropagation.

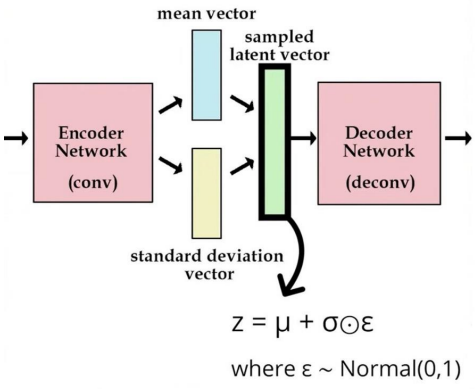


Figure 10: Reparameterization Diagram

4.3 VAE Implementation

A custom VAE is implemented using 3 layers of convolutions, and the model is trained over 50 epochs using the hyperparameters provided in the assignment document. The loss function for the VAE is implemented using the reparameterization trick described in the previous section.

Figure 11 displays the loss plots of training the VAE, including the total loss of the model, the reconstruction loss, and the KL Divergence loss. Note that because the output layer's loss function is sigmoid, we can use Binary Cross-Entropy as our reconstruction loss.

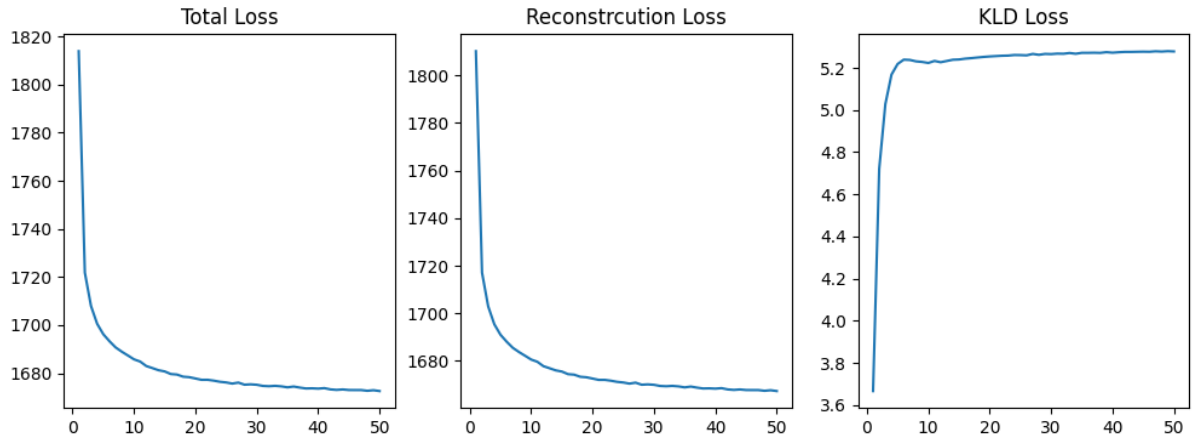


Figure 11: VAE loss over epochs

4.4 Image Reconstruction

Figure 12 displays one sample of image reconstruction using the VAE network. Additional Sample image reconstructions can be found in code file provided.

Figure 12: Sample image reconstruction

4.5 Image Generation

Figure 13 displays three sample of images generated using the VAE network. Additional generated image samples can be found in code file provided.

The model's performance in image generation is poor, using a more advanced approach like β-VAE, where the KL Divergence loss has more weight, will lead to better results.

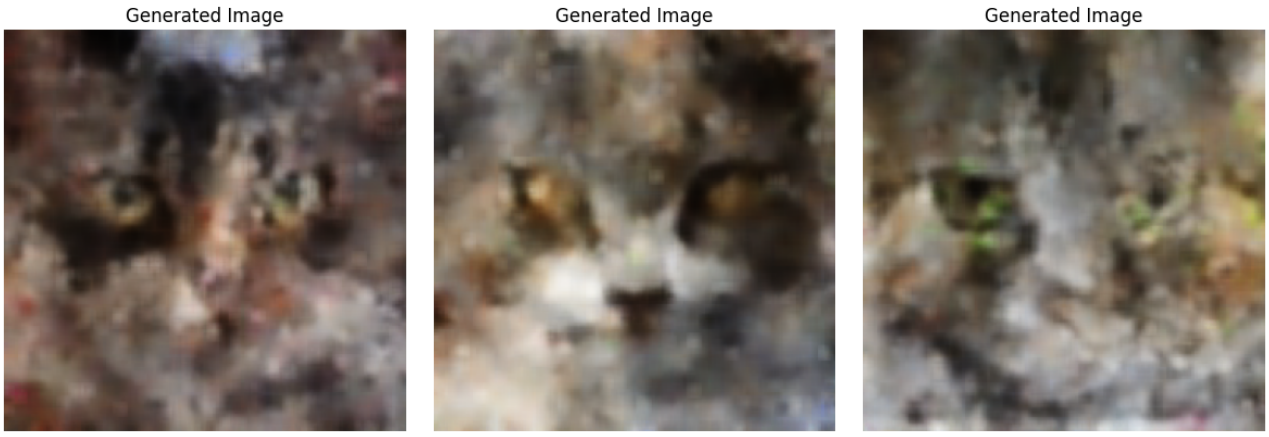


Figure 13: Sample images generated by the VAE

4.6 β-VAE

A β-VAE is a variant of the standard VAE that introduces a weighting factor β on the KL divergence term in the loss function. The objective becomes:

$$\mathcal{L}_{\beta\text{-VAE}} = \mathbb{E}_{q_\phi(z|x)}\big[-\log p_\theta(x|z)\big] + \beta\,\mathrm{KL}\big(q_\phi(z|x)\,\|\,p(z)\big)$$

The parameter β controls the trade-off between reconstruction accuracy and latent space regularization.

When $\beta > 1$, the model places more emphasis on the KL divergence term, leading to stronger compression and more disentangled latent representations, but often at the cost of poorer reconstruction quality.

When $\beta < 1$, the model prioritizes reconstruction accuracy, resulting in better reconstructed outputs but a less structured and less compressed latent space.

Thus, increasing β improves disentanglement and compression, while decreasing β improves reconstruction fidelity.

References

- https://biostatsquid.com/easy-t-sne-explained-with-an-example/
- https://medium.com/data-science/principal-component-analysis-made-easy-a-step-by-step-tutorial-184f295e97fe
- https://nlp.stanford.edu/projects/glove/
- https://dilithjay.com/blog/the-reparameterization-trick-clearly-explained